

QCGNN Project: Baseline vs Contributions

PART 1: The Pre-Existing Foundation (What was already there)

When I first inspected your workspace, you already had a robust, highly structured, and scientifically validated pipeline. You had successfully completed the data preprocessing, the causal extraction, and two primary experiments.

Layer 1 & Layer 2: Raw Data & Preprocessing

- What it does: Reads raw .xyz QM9 files and converts them into PyTorch Geometric (PyG) Data objects.
- Key Files: config.py, qcgnn_preprocessing.py, qm9_preprocess_dataset.py
- Existing State:
 - Extracted full molecular graphs (up to 18 atoms).
 - Generated 9-dimensional node features and 4-dimensional edge features.
 - Extracted exactly 3 targets: HOMO, LUMO, and GAP.
 - Applied rigorous Z-score normalization (target_mean, target_std) saved to data/qm9/layer2/norm_stats.pkl.
 - Outputs saved as full-graph datasets in data/qm9/layer2/ (qm9_train.pkl, etc.).

Layer 3: Causal Graph Extraction

- What it does: Uses a gradient-based causal inference model to strip away 'unimportant' atoms, reducing the full graphs to smaller, causally relevant subgraphs.
- Key Files: layer3_causal.py, code/layer3_qcgnn_model/qcgnn_causal_extractor.py
- Existing State:
 - Reduced molecular graphs from ~18 atoms down to a variable 3-12 atoms.
 - Preserved the original normalized targets perfectly.
 - Outputs saved as causal-graph datasets in data/qm9/layer3/.

Layer 4 / Experiment C: The Hybrid QCGNN Model

- What it does: A hybrid quantum-classical GNN running on the Causal graphs (Experiment C).
- Key Files: code/layer4_quantum/quantum_model.py, quantum_circuit.py, quantum_train.py
- Existing State:
 - A highly customized PennyLane + PyTorch architecture (HybridQuantumGNN).
 - Used Mean() + Max() pooling to convert variable graphs into a fixed 18-dim vector.
 - Projected into an 8-qubit quantum circuit using RY angle encoding, parameterized rotation layers, and CNOT entanglement.
 - Classical readout MLP to predict the 3 targets.
 - Evaluated and saved to models/best_qcgnn.pt.

Experiment A: Classical Baseline

- What it does: A purely classical PyG GENConv network running on the Full graphs.
- Key Files: code/baselines/classical_gnn/
- Existing State:
 - Acted as the classical benchmark against which quantum models would be compared.
 - Trained and saved to models/best_classical_gnn.pt.

QCGNN Project: Baseline vs Contributions

PART 2: What We Built Upon It (Our Additions)

Our primary goal was to complete a 2x2 Experimental Matrix (Classical vs. Quantum across Full vs. Causal graphs) without modifying or breaking any of your existing Layers 1, 2, 3, or 4. Here is exactly what we built into the workspace:

1. Built Experiment B (Causal Graph + Classical GENConv)

- Purpose: To determine if the performance of QCGNN (Exp C) was due to the quantum circuit or just the causal graph reduction.
- What we added (code/baselines/causal_classical_gnn/):
 - `causal_classical_gnn_model.py`: Replicated the 3-layer GENConv architecture from Experiment A.
 - `causal_classical_gnn_sanity_test.py`: A strict test confirming the classical model could process the variable-sized (3-12 atom) Layer 3 causal graphs.
 - `causal_classical_gnn_train.py` & `evaluate.py`: Custom training loops pointing exclusively to Layer 3 .pkl files and saving to `best_causal_classical_gnn.pt`.

2. Built Experiment D (Full Graph + Hybrid QCGNN)

- Purpose: To determine how the QCGNN behaves on complete molecules without causal reduction, providing a direct comparison to Experiment A.
- What we added (code/experiments/experiment_D_full_qcgnn/):
 - `full_qcgnn_model.py`: A smart wrapper that directly imports your validated HybridQuantumGNN.
 - `full_qcgnn_sanity_test.py`: Verified that the Mean() + Max() pooling flawlessly handles the larger 18-atom graphs from Layer 2.
 - `full_qcgnn_train.py` & `evaluate.py`: Scripts to train the quantum model on Layer 2 data, independently saving to `best_full_qcgnn.pt`.

3. Rewrote the Visualization Engine

- Purpose: We needed a script capable of automatically parsing all 4 experiments and generating comparative graphics for a research paper.
- What we added (Updated code/visualization/visualize_results.py):
 - Added dynamic loading to safely check which of the 4 experiments (A, B, C, D) have completed.
 - Automated the creation of a clean folder structure (results/visualizations/experiment_X/).
 - Programmed comparative bar charts (MAE, RMSE, and R2).
 - Generated a unified `model_comparison.csv` aggregating all experiments.

4. Conducted the Multi-Task Expansion Audit (3 -> 6 Targets)

- Purpose: You requested an investigation into expanding the project from 3 targets to 6 targets to improve the scientific scope.
- What we added (Feasibility Report):
 - Built zero new code for this yet (per your instructions), but executed background scripts analyzing your raw .xyz files.
 - Verified that 15 targets physically exist in your raw data.
 - Identified and proposed the 3 best complementary targets: Dipole Moment (μ), Isotropic

QCGNN Project: Baseline vs Contributions

Polarizability (α), and Heat Capacity (c_v).

° Laid down the exact architectural roadmap for how to implement this across Layers 2, 3, and 4 (including fixing the evaluation scripts).